

Negative Sampling Loss for Cross Entropy Error with Cauchy Kernel:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

def phi(x,y,a,b):
    '''Kernel Function'''
    factor = 1 + a * torch.sum((x-y)**2) ** b
    y = 1/factor

    return y

def criterion_pos(x,y,a,b):
    '''Positive Loss'''
    prob = phi(x,y,a,b)
    y = -torch.log(prob)

    return y

def criterion_neg(x,y,a,b):
    '''Negative Loss'''
    prob = phi(x,y,a,b)
    y = -torch.log(1-prob+0.0001)

    return y
```

Feedforward Network:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

class network(nn.Module):
    def __init__(self, channels=[2,100,100,100,100,100,100,100,100,2], input_size=None):

        super(network, self).__init__()
        weights = []
        biases = []
        gammas = []

        N = len(channels)

        self.layers = N-1

        if input_size is not None:
            channels[0] = input_size
```

```

for i in range(self.layers):
    w = torch.as_tensor(0.5*np.random.randn(channels[i],channels[i+1]), dtype=torch.float32)
    weights.append(nn.Parameter(w))
    biases.append(nn.Parameter(torch.zeros((channels[i+1]))))
    gammas.append(nn.Parameter(torch.ones((channels[i+1]))))

self.weights = nn.ParameterList(weights)
self.biases = nn.ParameterList(biases)
self.gammas = nn.ParameterList(gammas)

self.nl = torch.sigmoid

def forward(self, inp):

    for i in range(self.layers-1):
        #print(i, inp)
        x_1 = torch.mm(inp,self.weights[i])
        #print(x_1)
        x_2 = x_1 + self.biases[i]
        #print(x_2)
        x_2 = x_2 * self.gammas[i]
        #print(x_2)
        inp = self.nl(x_2)

    x_1 = torch.mm(inp,self.weights[self.layers-1]) + self.biases[self.layers-1]
    out = x_1 * self.gammas[self.layers-1]

    return out

```